



EXAMEN DELIVERUS - FRONTEND

(MODELO F: COMISIONES)

Enunciado del examen Se ha de implementar la interfaz gráfica de los requisitos funcionales de los propietarios relacionados con la nueva característica de **Comisiones**. En el backend ya se implementó que cada restaurante pertenece a un plan de comisiones (Gratis o De Pago) y que se puede consultar la deuda acumulada.

Requisitos Funcionales a implementar:

RF.01 - Asignar comisión a un restaurante (Creación y Edición) Como propietario, quiero asignar un plan de comisión al crear o editar mis restaurantes, para controlar los costes del servicio. *Pruebas de aceptación:*

- En los formularios de creación (`CreateRestaurantScreen`) y edición (`EditRestaurantScreen`), debe aparecer un campo desplegable (Select) obligatorio para elegir el plan de comisión.
- Los planes de comisión deben cargarse desde la base de datos (endpoint `GET /commissions`).
- Si al intentar guardar se incumple la regla de negocio (ej. ya tienes un restaurante gratuito y eliges el plan gratuito para otro), se debe mostrar un mensaje de error capturando el código 409 del backend.

RF.02 - Consultar Comisión Acumulada Como propietario, quiero consultar la comisión acumulada de mi restaurante, para conocer los costes totales del servicio que le debo a la plataforma. *Pruebas de aceptación:*

- En la pantalla de detalle del restaurante (`RestaurantDetailScreen`), se debe mostrar una nueva sección o tarjeta que indique la **Deuda Acumulada** del restaurante.
- El valor debe obtenerse llamando al endpoint `GET /restaurants/:restaurantId/commission` que devuelve el JSON `{ "totalCommission": X.XX }`.



SOLUCIÓN DEL EXAMEN (CÓDIGOS)

Para resolver este examen, tú (como estudiante) tendrías que modificar 3 cosas principales: crear el endpoint, modificar la pantalla de detalle y modificar el formulario. Aquí tienes cómo se haría para sacar un 10.

1. Crear las llamadas a la API (**src/api/CommissionEndpoints.js**)

Lo primero de todo es crear un archivo (o añadirlo a uno existente) para poder comunicarnos con los endpoints que hiciste en tu examen de backend.

JavaScript

```
// src/api/CommissionEndpoints.js
import { get } from './helpers/ApiRequestsHelper'

// Para llenar el desplegable de los formularios (RF1)
function getCommissions() {
  return get('commissions')
}

// Para obtener la deuda acumulada de un restaurante concreto (RF2)
function getAccumulatedCommission(restaurantId) {
  return get(`restaurants/${restaurantId}/commission`)
}

export { getCommissions, getAccumulatedCommission }
```

2. Modificar el Detalle del Restaurante (RF.02)

Vamos a **src/screens/restaurants/RestaurantDetailScreen.js** para añadir la llamada a la API y pintar el cuadrito con la deuda acumulada.

```
import React, { useEffect, useState } from 'react'
import { StyleSheet, View, FlatList, ImageBackground, Image } from 'react-native'
import { showMessage } from 'react-native-flash-message'
import { getDetail } from '../api/RestaurantEndpoints'
import { getAccumulatedCommission } from '../api/CommissionEndpoints' // <-- IMPORTAMOS EL NUEVO ENDPOINT
import TextRegular from '../components/TextRegular'
import TextSemiBold from '../components/TextSemiBold'
import * as GlobalStyles from '../styles/GlobalStyles'
import { API_BASE_URL } from '@env'

export default function RestaurantDetailScreen({ route }) {
  const [restaurant, setRestaurant] = useState({})
  const [commissionDebt, setCommissionDebt] = useState(0) // <-- ESTADO PARA LA DEUDA

  useEffect(() => {
    fetchRestaurantDetail()
    fetchCommissionData() // <-- LLAMAMOS A LA DEUDA AL CARGAR LA PANTALLA
  }, [route])
```

```

const fetchRestaurantDetail = async () => {
  try {
    const fetchedRestaurant = await getDetail(route.params.id)
    setRestaurant(fetchedRestaurant)
  } catch (error) {
    showMessage({ message: 'Error fetching details', type: 'error' })
  }
}

const fetchCommissionData = async () => {
  try {
    // Usamos el ID del restaurante que viene por parámetros en la navegación
    const data = await getAccumulatedCommission(route.params.id)
    setCommissionDebt(data.totalCommission)
  } catch (error) {
    console.log('Error fetching commission data', error)
  }
}

const renderHeader = () => {
  return (
    <View>
      <ImageBackground
        source={restaurant?.heroImage ? { uri: API_BASE_URL + '/' + restaurant.heroImage }
: undefined}
        style={styles.imageBackground}
      >
        <View style={styles.restaurantHeaderContainer}>
          <TextSemiBold textStyle={styles.textTitle}>{restaurant.name}</TextSemiBold>
          <TextRegular textStyle={styles.description}>{restaurant.description}</TextRegular>
        </View>
      </ImageBackground>

      { /* NUEVA SECCIÓN DE COMISIONES (RF2) */ }
      <View style={styles.commissionCard}>
        <TextSemiBold textStyle={{ fontSize: 16 }}>Financial Overview</TextSemiBold>
        <TextRegular>Accumulated Debt: {commissionDebt.toFixed(2)} €</TextRegular>
      </View>

    </View>
  )
}

// ... (El resto del código con el FlatList y los productos se queda igual)

return (
  <FlatList

```

```

    ListHeaderComponent={renderHeader}
    style={styles.container}
    data={restaurant.products}
    keyExtractor={item => item.id.toString()}
    renderItem={/* tu renderProduct habitual */}
  />
)
}

const styles = StyleSheet.create({
  // ... (Tus estilos habituales)
  commissionCard: {
    backgroundColor: '#ffebee', // Un color rojizo para que destaque
    padding: 15,
    margin: 10,
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#ffcdd2',
    alignItems: 'center'
  }
})

```

3. Modificar el Formulario de Edición (RF.01)

Aquí es donde usarías **Formik** y **Yup** para añadir el campo de la comisión en `src/screens/restaurants/EditRestaurantScreen.js`. (El proceso es idéntico para `CreateRestaurantScreen`).

```

import React, { useEffect, useState } from 'react'
import { StyleSheet, View, ScrollView } from 'react-native'
import * as yup from 'yup'
import { Formik } from 'formik'
import { showMessage } from 'react-native-flash-message'
import DropDownPicker from 'react-native-dropdown-picker' // Componente habitual en DeliverUS
import { update, getDetail } from '../api/RestaurantEndpoints'
import { getCommissions } from '../api/CommissionEndpoints' // <-- IMPORTAMOS EL ENDPOINT
import InputItem from '../components/InputItem'
import TextRegular from '../components/TextRegular'
import * as GlobalStyles from '../styles/GlobalStyles'

export default function EditRestaurantScreen({ navigation, route }) {
  const [restaurant, setRestaurant] = useState(null)

  // Estados para el desplegable de Comisiones
  const [commissions, setCommissions] = useState([])

```

```
const [open, setOpen] = useState(false)
```

```
useEffect(() => {  
  async function fetchData() {  
    try {  
      const fetchedRestaurant = await getDetail(route.params.id)  
      setRestaurant(fetchedRestaurant)  
  
      // Cargar las comisiones disponibles para el desplegable  
      const fetchedCommissions = await getCommissions()  
      // Mapear los datos al formato que necesita el DropDownPicker {label, value}  
      const commissionItems = fetchedCommissions.map(c => ({  
        label: `${c.name} (${c.percentage}%}`,  
        value: c.id  
      })))  
      setCommissions(commissionItems)  
    } catch (error) {  
      showMessage({ message: 'Error fetching data', type: 'error' })  
    }  
  }  
  fetchData()  
}, [route])
```

```
// Añadimos el commissionId a la validación  
const validationSchema = yup.object().shape({  
  name: yup.string().required('Name is required'),  
  // ... (resto de validaciones)  
  commissionId: yup.number().positive('Please select a commission  
plan').required('Required')  
})
```

```
if (!restaurant) return null
```

```
return (  
  <Formik  
    initialValues={{  
      name: restaurant.name,  
      // ... (resto de valores)  
      commissionId: restaurant.commissionId || " " // <-- AÑADIMOS ESTO  
    }}  
    validationSchema={validationSchema}  
    onSubmit={async (values) => {  
      try {  
        await update(restaurant.id, values)  
        showMessage({ message: 'Restaurant updated!', type: 'success' })  
        navigation.goBack()  
      } catch (error) {  
        // Si tu backend devuelve un 409 por la regla del plan gratuito, caerá aquí
```

```

        showMessage({
            message: `Could not update: ${error.message}`,
            type: 'error'
        })
    }
}
}
>
    ({ handleChange, handleBlur, handleSubmit, setFieldValue, values, errors }) => (
        <ScrollView style={{ padding: 20 }}>
            {/* Inputs habituales */}
            <InputItem name="name" value={values.name}
onChangeText={handleChange('name')} error={errors.name} />

            {/* NUEVO: DESPLEGABLE DE COMISIONES (RF1) */}
            <TextRegular style={styles.label}>Commission Plan</TextRegular>
            <DropDownPicker
                open={open}
                value={values.commissionId}
                items={commissions}
                setOpen={setOpen}
                setValue={(val) => setFieldValue('commissionId', val())}
                placeholder="Select a commission plan"
                style={styles.dropdown}
                listMode="SCROLLVIEW"
            />
            {errors.commissionId && <TextRegular
style={styles.error}>{errors.commissionId}</TextRegular>}

            {/* Botón de guardar */}
            <Pressable onPress={handleSubmit} style={styles.button}>
                <TextRegular textStyle={{ color: 'white' }}>Save</TextRegular>
            </Pressable>
        </ScrollView>
    )
}

const styles = StyleSheet.create({
    label: { marginBottom: 5, fontSize: 16 },
    dropdown: { marginBottom: 15, borderColor: '#ccc' },
    error: { color: 'red', marginBottom: 10 },
    button: { backgroundColor: GlobalStyles.brandPrimary, padding: 15, borderRadius: 8,
alignItems: 'center' }
})

```



Análisis del simulacro para tu examen de mañana:

Fíjate en el patrón porque **siempre es el mismo**:

1. **Llamadas a la API:** Creas funciones pequeñas en la carpeta `api/` usando `get`, `post`, `put` o `delete`.
2. **Listar datos (GET):** En la pantalla, usas `useEffect` con un array vacío `[]` para llamar a la API y guardas el resultado con `useState`. Luego lo pintas con un `<FlatList>` o en un `<Text>`.
3. **Manejar Formularios (POST/PUT):** Usas `<Formik>`. Le das los `initialValues`, el `validationSchema` con Yup, y en el `onSubmit` llamas a tu API. Si la API falla (ej. error 409 de las reglas de tu backend), el bloque `catch` hace saltar un `showMessage` rojo.

Con esto que acabas de ver y los apuntes anteriores, vas sobradísimo para mañana.
¡Mucha suerte!